

cw_ml_for_imaging-7

February 21, 2024

1 Coursework: Self-supervised learning

In this coursework, you will explore the popular self-supervised contrastive learning approach [SimCLR](#).

You will be asked to implement some of the key components of SimCLR, including a suitable data augmentation strategy (for generating positive pairs), the SimCLR loss function, and the SimCLR training step. Additionally, you will be using transfer learning strategies for evaluating the performance of different pre-trained models for a downstream classification task.

The coursework is divided into three-parts: - **Part A:** Implementation of a suitable dataset for contrastive model training; - **Part B:** Implementation of the SimCLR loss and training step; - **Part C:** Implementation of transfer learning strategies (linear probing and finetuning) for model evaluation.

Important: Read the text descriptions carefully and look out for hints and comments indicating a specific ‘TASK’. Make sure to add sufficient documentation to your code.

Submission: You are asked to submit two versions of your notebook: 1. You should submit the raw notebook in `.ipynb` format with *all outputs cleared*. Please name your file `coursework.ipynb`. 2. Additionally, you will be asked to submit an exported version of your notebook in `.pdf` format, with *all outputs included*. We will primarily use this version for marking, but we will use the raw notebook to check for correct implementations. Please name this file `coursework_export.pdf`.

1.1 Your details

Please add your details below. You can work in groups up to two.

Authors: **Piotr Blaszyk & Lizzie Williams**

DoC alias: **psb120 & lsw120**

1.2 Setup

```
[1]: # On Google Colab uncomment the following line to install PyTorch Lightning and
    ↪ the MedMNIST dataset
    ! pip install lightning medmnist
```

Collecting lightning

Downloading lightning-2.2.0.post0-py3-none-any.whl (2.1 MB)

2.1/2.1 MB

15.5 MB/s eta 0:00:00

Collecting medmnist

Downloading medmnist-3.0.1-py3-none-any.whl (25 kB)

Requirement already satisfied: PyYAML<8.0,>=5.4 in

/usr/local/lib/python3.10/dist-packages (from lightning) (6.0.1)

Requirement already satisfied: fsspec[http]<2025.0,>=2022.5.0 in

/usr/local/lib/python3.10/dist-packages (from lightning) (2023.6.0)

Collecting lightning-utilities<2.0,>=0.8.0 (from lightning)

Downloading lightning_utilities-0.10.1-py3-none-any.whl (24 kB)

Requirement already satisfied: numpy<3.0,>=1.17.2 in

/usr/local/lib/python3.10/dist-packages (from lightning) (1.25.2)

Requirement already satisfied: packaging<25.0,>=20.0 in

/usr/local/lib/python3.10/dist-packages (from lightning) (23.2)

Requirement already satisfied: torch<4.0,>=1.13.0 in

/usr/local/lib/python3.10/dist-packages (from lightning) (2.1.0+cu121)

Collecting torchmetrics<3.0,>=0.7.0 (from lightning)

Downloading torchmetrics-1.3.1-py3-none-any.whl (840 kB)

840.4/840.4

kB 29.0 MB/s eta 0:00:00

Requirement already satisfied: tqdm<6.0,>=4.57.0 in

/usr/local/lib/python3.10/dist-packages (from lightning) (4.66.2)

Requirement already satisfied: typing-extensions<6.0,>=4.4.0 in

/usr/local/lib/python3.10/dist-packages (from lightning) (4.9.0)

Collecting pytorch-lightning (from lightning)

Downloading pytorch_lightning-2.2.0.post0-py3-none-any.whl (800 kB)

800.9/800.9

kB 36.1 MB/s eta 0:00:00

Requirement already satisfied: pandas in /usr/local/lib/python3.10/dist-packages (from medmnist) (1.5.3)

Requirement already satisfied: scikit-learn in /usr/local/lib/python3.10/dist-packages (from medmnist) (1.2.2)

Requirement already satisfied: scikit-image in /usr/local/lib/python3.10/dist-packages (from medmnist) (0.19.3)

Requirement already satisfied: Pillow in /usr/local/lib/python3.10/dist-packages (from medmnist) (9.4.0)

Collecting fire (from medmnist)

Downloading fire-0.5.0.tar.gz (88 kB)

88.3/88.3 kB

13.7 MB/s eta 0:00:00

Preparing metadata (setup.py) ... done

Requirement already satisfied: torchvision in /usr/local/lib/python3.10/dist-packages (from medmnist) (0.16.0+cu121)

Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from fsspec[http]<2025.0,>=2022.5.0->lightning) (2.31.0)

Requirement already satisfied: aiohttp!=4.0.0a0,!4.0.0a1 in

/usr/local/lib/python3.10/dist-packages (from fsspec[http]<2025.0,>=2022.5.0->lightning) (3.9.3)

Requirement already satisfied: setuptools in /usr/local/lib/python3.10/dist-packages (from lightning-utilities<2.0,>=0.8.0->lightning) (67.7.2)

Requirement already satisfied: filelock in /usr/local/lib/python3.10/dist-packages (from torch<4.0,>=1.13.0->lightning) (3.13.1)

Requirement already satisfied: sympy in /usr/local/lib/python3.10/dist-packages (from torch<4.0,>=1.13.0->lightning) (1.12)

Requirement already satisfied: networkx in /usr/local/lib/python3.10/dist-packages (from torch<4.0,>=1.13.0->lightning) (3.2.1)

Requirement already satisfied: Jinja2 in /usr/local/lib/python3.10/dist-packages (from torch<4.0,>=1.13.0->lightning) (3.1.3)

Requirement already satisfied: triton==2.1.0 in /usr/local/lib/python3.10/dist-packages (from torch<4.0,>=1.13.0->lightning) (2.1.0)

Requirement already satisfied: six in /usr/local/lib/python3.10/dist-packages (from fire->medmnist) (1.16.0)

Requirement already satisfied: termcolor in /usr/local/lib/python3.10/dist-packages (from fire->medmnist) (2.4.0)

Requirement already satisfied: python-dateutil>=2.8.1 in /usr/local/lib/python3.10/dist-packages (from pandas->medmnist) (2.8.2)

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas->medmnist) (2023.4)

Requirement already satisfied: scipy>=1.4.1 in /usr/local/lib/python3.10/dist-packages (from scikit-image->medmnist) (1.11.4)

Requirement already satisfied: imageio>=2.4.1 in /usr/local/lib/python3.10/dist-packages (from scikit-image->medmnist) (2.31.6)

Requirement already satisfied: tifffile>=2019.7.26 in /usr/local/lib/python3.10/dist-packages (from scikit-image->medmnist) (2024.2.12)

Requirement already satisfied: PyWavelets>=1.1.1 in /usr/local/lib/python3.10/dist-packages (from scikit-image->medmnist) (1.5.0)

Requirement already satisfied: joblib>=1.1.1 in /usr/local/lib/python3.10/dist-packages (from scikit-learn->medmnist) (1.3.2)

Requirement already satisfied: threadpoolctl>=2.0.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn->medmnist) (3.2.0)

Requirement already satisfied: aiohttp>=1.1.2 in /usr/local/lib/python3.10/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]<2025.0,>=2022.5.0->lightning) (1.3.1)

Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]<2025.0,>=2022.5.0->lightning) (23.2.0)

Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.10/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]<2025.0,>=2022.5.0->lightning) (1.4.1)

Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.10/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]<2025.0,>=2022.5.0->lightning) (6.0.5)

Requirement already satisfied: yarl<2.0,>=1.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]<2025.0,>=2022.5.0->lightning) (1.9.4)

Requirement already satisfied: async-timeout<5.0,>=4.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]<2025.0,>=2022.5.0->lightning) (4.0.3)

Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.10/dist-packages (from jinja2->torch<4.0,>=1.13.0->lightning) (2.1.5)

Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->fsspec[http]<2025.0,>=2022.5.0->lightning) (3.3.2)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests->fsspec[http]<2025.0,>=2022.5.0->lightning) (3.6)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests->fsspec[http]<2025.0,>=2022.5.0->lightning) (2.0.7)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests->fsspec[http]<2025.0,>=2022.5.0->lightning) (2024.2.2)

Requirement already satisfied: mpmath>=0.19 in /usr/local/lib/python3.10/dist-packages (from sympy->torch<4.0,>=1.13.0->lightning) (1.3.0)

Building wheels for collected packages: fire

Building wheel for fire (setup.py) ... done

Created wheel for fire: filename=fire-0.5.0-py2.py3-none-any.whl size=116934 sha256=72dda0f83ab6d48c453bb81771e849531b25b30f01c2510c1f2a1fccfc61bb4b

Stored in directory: /root/.cache/pip/wheels/90/d4/f7/9404e5db0116bd4d43e5666eaa3e70ab53723e1e3ea40c9a95

Successfully built fire

Installing collected packages: lightning-utilities, fire, torchmetrics, pytorch-lightning, medmnist, lightning

Successfully installed fire-0.5.0 lightning-2.2.0.post0 lightning-utilities-0.10.1 medmnist-3.0.1 pytorch-lightning-2.2.0.post0 torchmetrics-1.3.1

```
[39]: import os
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision
import matplotlib.pyplot as plt

from torch import linalg as LA
from torch.utils.data import DataLoader
from torchsummary import summary
from torchvision import models
from torchvision import transforms
from torchvision.transforms import v2
from pytorch_lightning import LightningModule, LightningDataModule, Trainer,
↳seed_everything
```

```

from pytorch_lightning.loggers import TensorBoardLogger
from pytorch_lightning.callbacks import ModelCheckpoint, TQDMProgressBar
from torchmetrics.functional import auroc, accuracy
from PIL import Image
from medmnist.info import INFO
from medmnist.dataset import MedMNIST

```

1.3 Part A: Implement a dataset suitable for contrastive learning.

We will be using the [MedMNIST Pneumonia](#) dataset, which is a medical imaging inspired dataset but with the characteristics of MNIST. This allows efficient experimentation due to the small image size. The dataset contains real chest X-ray images but downsampled to 28 x 28 pixels, with binary labels indicating the presence of [Pneumonia](#) (which is an inflammation of the lungs).

1.3.1 Task A-1: Complete the dataset implementation.

You are asked to implement a dataset class `SimCLRPNPneumoniaMNISTDataset` suitable for training a self-supervised model with a contrastive objective. For each sample, your dataset class should return two ‘views’ of the corresponding image, forming the positive pairs for contrastive learning. It is up to you to design suitable augmentation pipeline for generating these views. Please provide a short description in plain language of what your data augmentation pipeline is meant to do.

To get you started, we have provided the skeleton of the dataset class in the cell below. Once you have implemented your dataset class, you are asked to run the provided visualisation code to visualise one batch of your training dataloader.

Note: You can use the same data augmentation pipeline for training, validation, and testing.

```

[40]: class SimCLRPNPneumoniaMNISTDataset(MedMNIST):
    def __init__(self, split = 'train'):
        ''' Dataset class for PneumoniaMNIST.
            The provided init function will automatically download the necessary
            files at the first class initialisation.

            :param split: 'train', 'val' or 'test', select subset

            '''
        self.flag = "pneumoniamnist"
        self.size = 28
        self.size_flag = ""
        self.root = './data/coursework/'
        self.info = INFO[self.flag]
        self.download()

        npz_file = np.load(os.path.join(self.root, "pneumoniamnist.npz"))

        self.split = split

        # Load all the images

```

```

assert self.split in ['train','val','test']

self.imgs = npz_file[f'{self.split}_images']
self.labels = npz_file[f'{self.split}_labels']

# TASK: Define here your data augmentation pipeline
# Add a short description in plain language.
"""
I add augmentations for features that vary between images.
I want the model to disregard these features.
How much each features varies was determined via manual inspection
of the 8 random image pairs from the visualisation cell below.
Features that vary:
* Zoom - hence RandomResizedCrop
* Average pixel intensity - some images are brighter than others
  - hence ColorJitter
* Rotation - hence RandomRotation
Overall, the variance of each of the above features is rather small,
hence the augmentations are also small.

Augmentations are not applied to the features that I want the model
to use. Those features remain the same across all images in each class.
Some constant features (and related augmentations) include:
* Vertical and horizontal flip - assume the patient always lies
  on their back with their head pointing up
* Cutout - assume nothing is obscuring the view of the lungs
  in the scans
* Sobel filtering - assume the scans are of high enough quality
  that more than just the contour is visible
"""

self.augmentation_pipeline = v2.Compose([
    v2.RandomResizedCrop(size=28, scale=(0.5, 1.0), antialias=True),
    v2.ColorJitter(brightness=0.15,
                  contrast=None,
                  saturation=None,
                  hue=None),
    v2.RandomRotation(degrees=15),

    v2.ToImage(),
    v2.ToDtype(torch.float32, scale=True),
])

def __len__(self):
    return self.imgs.shape[0]

def __getitem__(self, index):
    # TASK: Fill in the blanks such that you return two tensors

```

```

        # of shape [1, 28, 28], img_view1 and img_view2, representing two
        ↪ augmented view of the images.
        img = self.imgs[index]
        img = img.reshape((1, 28, 28))
        img = torch.from_numpy(img)
        img_view1 = self.augmentation_pipeline(img)
        img_view2 = self.augmentation_pipeline(img)
        return img_view1, img_view2

```

We use a [LightningDataModule](#) for handling your PneumoniaMNIST dataset. You do not need to make any modifications to the code below.

```

[41]: class SimCLRPneumoniaMNISTDataModule(LightningDataModule):
        def __init__(self, batch_size: int = 8):
            super().__init__()
            self.batch_size = batch_size
            self.train_set = SimCLRPneumoniaMNISTDataset(split='train')
            self.val_set = SimCLRPneumoniaMNISTDataset(split='val')
            self.test_set = SimCLRPneumoniaMNISTDataset(split='test')

        def train_dataloader(self):
            return DataLoader(dataset=self.train_set, batch_size=self.batch_size,
            ↪ shuffle=True)

        def val_dataloader(self):
            return DataLoader(dataset=self.val_set, batch_size=self.batch_size,
            ↪ shuffle=False)

        def test_dataloader(self):
            return DataLoader(dataset=self.test_set, batch_size=self.batch_size,
            ↪ shuffle=False)

```

Check dataset implementation. Run the below cell to visualise a batch of your training dataloader.

```

[42]: # DO NOT MODIFY THIS CELL! IT IS FOR CHECKING THE IMPLEMENTATION ONLY.

# Initialise data module
datamodule = SimCLRPneumoniaMNISTDataModule()
# Get train dataloader
train_dataloader = datamodule.train_dataloader()
# Get first batch
batch = next(iter(train_dataloader))
# Visualise the images
view1, view2 = batch
f, ax = plt.subplots(2, 8, figsize=(12,4))
for i in range(8):

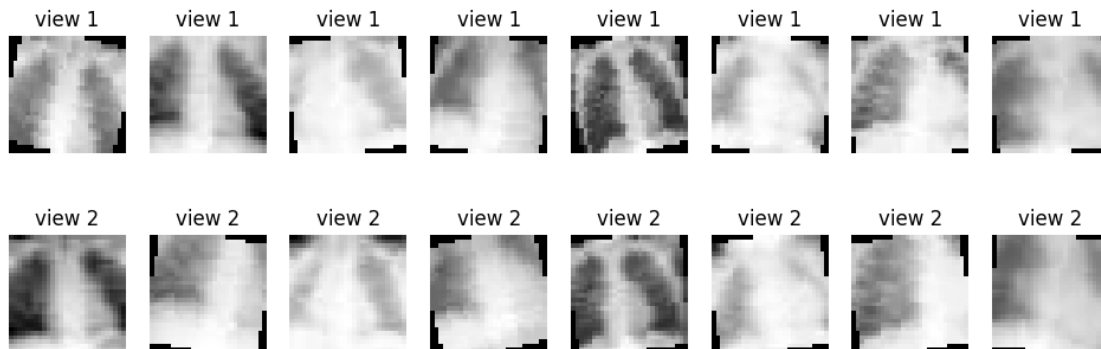
```

```
ax[0,i].imshow(view1[i, 0], cmap='gray')
ax[1,i].imshow(view2[i, 0], cmap='gray')
ax[0,i].set_title('view 1')
ax[1,i].set_title('view 2')
ax[0, i].axis("off")
ax[1, i].axis("off")
```

Using downloaded and verified file: ./data/coursework/pneumoniarnist.npz

Using downloaded and verified file: ./data/coursework/pneumoniarnist.npz

Using downloaded and verified file: ./data/coursework/pneumoniarnist.npz



1.4 Part B: Implement the SimCLR loss and training step.

In this part, we ask you to: 1. Implement the SimCLR loss function, as per the equation in the lecture notes (and the [original paper](#)). 2. Once you have implemented the loss, implement the training step function in the provided LightningModule.

1.4.1 Task B-1: SimCLR loss function.

For the implementation of the SimCLR loss, you should follow the ‘recipe’ from the lecture slides. We provide a code skeleton to get you started. Fill in all the blanks.

Hint: In PyTorch, to compute scalar products (also called dot products) between many elements efficiently, note that for two batches of d -dimensional feature vectors $v1$ and $v2$ of size $[N, d]$ (with N being the batch size) computing the matrix multiplication `torch.mm(v1, v2.t())` returns a matrix S of size $[N, N]$ where each element $S[i, j]$ is the scalar product of $v1_i$ and $v2_j$.

```
[43]: def simclr_loss(embedding_view1, embedding_view2, tau = 1.0):
    '''
    This funtion implements the SimCLR loss function as described in the original_
    ↪paper.
    See lecture notes for formulas.

    It takes as input the embeddings from both views and returns the loss value_
    ↪for that batch.
```

```

Args:
    embedding_view1: torch tensor of shape [batch_size, embedding_dimension]
    embedding_view2: torch tensor of shape [batch_size, embedding_dimension]
Returns:
    loss: torch.tensor of shape 1
'''

# Step 1: normalise the embeddings
view1_norm = LA.vector_norm(embedding_view1, dim=1)
view2_norm = LA.vector_norm(embedding_view2, dim=1)

view1_norm = view1_norm.to(embedding_view1.device)
view2_norm = view2_norm.to(embedding_view1.device)

embedding_view1 = (embedding_view1.t() / view1_norm).t()
embedding_view2 = (embedding_view2.t() / view2_norm).t()

# Step 2: gather all embeddings into one big vector of size [2*N ,
↪feature_dim]
z_all_views = torch.cat([embedding_view1, embedding_view2], dim=0)

z_all_views = z_all_views.to(embedding_view1.device)

# Step 3: compute all possible similarities, should be a matrix of size [2 *
↪N, 2 * N]
# all_similarities[i,j] will be the similarity between z_all_views[i] and
↪z_all_views[j].
# Use the hint.
all_similarities = torch.mm(z_all_views, z_all_views.t())

all_similarities = all_similarities.to(embedding_view1.device)

# Step 4: Here we want to return a mask of size[2 * N, 2 * N] for which
↪mask[i,j] = 1 if
# z_all_views[i] and z_all_views[j] form a positive pair.
# There should be exactly 2 * N non-zeros elements in this matrix.
n = list(all_similarities.shape)[0] // 2
mask = torch.zeros((2 * n, 2 * n), dtype=torch.int)
mask = mask.fill_diagonal_(1)
mask = torch.roll(mask, shifts=n, dims=0)

mask = mask.to(embedding_view1.device)

# Step 5: self-mask. For computing the denominator term in the loss function,
# we need to sum over all possible similarities except the self-similarity.
# Create a mask of shape [2*N, 2*N] that is 1 for all valid pairs and 0 for
↪all self-pairs (i = j).

```

```

self_mask = torch.ones((2 * n, 2 * n), dtype=torch.int)
self_mask = self_mask.fill_diagonal_(0)

self_mask = self_mask.to(embedding_view1.device)

# Step 6: Computing all numerators for the loss function.
# Should be vector of size [2 * N],
# where element is exp(sim(i, j) / t) for each positive pair (i, j).
# Re-use the computed quantities above.
positive_modified = torch.exp(all_similarities / tau) * mask
positive_modified = positive_modified.to(embedding_view1.device)

numerators = torch.sum(positive_modified, dim=1)
numerators = numerators.to(embedding_view1.device)

# Step 7: Computing all denominators for the loss function.
# Should be a vector of size [2 * N].
# Where each element should be the sum of exp(sim(i,k)/tau) for all k != i.
non_self_modified = torch.exp(all_similarities / tau) * self_mask
non_self_modified = non_self_modified.to(embedding_view1.device)

denominators = torch.sum(non_self_modified, dim=1)
denominators = denominators.to(embedding_view1.device)

# Step 8: Return the final loss values, using the previously computed
numerators and denominators.
all_loss = - torch.log(numerators / denominators)
all_loss = all_loss.to(embedding_view1.device)

batch_loss = torch.mean(all_loss)

return batch_loss

```

Check SimCLR loss function. To check your implementation, please run the following tests. Note that we will also use other tests on different inputs to test your code.

```

[44]: # DO NOT MODIFY THIS CELL! IT IS FOR CHECKING THE IMPLEMENTATION ONLY.

seed_everything(33)

expected_results = [torch.tensor(1.7518), torch.tensor(1.6376), torch.tensor(4.
↳194), torch.tensor(4.1754)]
for i, (N, feature_dim) in enumerate(zip([3, 3, 33, 33], [5, 125, 5, 125])):
    embedding_view1 = torch.rand((N, feature_dim))
    embedding_view2 = torch.rand((N, feature_dim))
    loss = simclr_loss(embedding_view1.clone(), embedding_view2.clone(), tau=0.5)
    print(f"Expected loss: {expected_results[i]}, Computed loss: {loss}")

```

```

    assert torch.isclose(loss, expected_results[i], rtol=1e-3)
    print("Passed all tests successfully !")

```

INFO:lightning_fabric.utilities.seed:Seed set to 33

Expected loss: 1.7517999410629272, Computed loss: 1.7518142461776733

Expected loss: 1.6375999450683594, Computed loss: 1.6376436948776245

Expected loss: 4.193999767303467, Computed loss: 4.194351673126221

Expected loss: 4.1753997802734375, Computed loss: 4.175372123718262

Passed all tests successfully !

1.4.2 Task B-2: SimCLR training step.

In this next task you are asked to complete the blanks in the provided [LightningModule](#).

We provide the implementation of an image encoder (the CNN backbone that will act as feature extractor). No changes are needed for this part.

```

[45]: class ImageEncoder(torch.nn.Module):
        def __init__(self) -> None:
            super().__init__()
            self.net = models.resnet50(weights=None)
            del self.net.fc
            self.net.conv1 = torch.nn.Conv2d(1, 64, kernel_size=7, stride=2,
            ↪padding=3, bias=False)

        def forward(self, x: torch.Tensor) -> torch.Tensor:
            x = self.net.conv1(x)
            x = self.net.bn1(x)
            x = self.net.relu(x)
            x0 = self.net.maxpool(x)
            x1 = self.net.layer1(x0)
            x2 = self.net.layer2(x1)
            x3 = self.net.layer3(x2)
            x4 = self.net.layer4(x3)
            x4 = self.net.avgpool(x4)
            x4 = torch.flatten(x4, 1)
            return x4

```

Next, you will need to complete the implementation of the SimCLR model. In order to make the training step work correctly, you will need to implement the `process_batch` function.

```

[46]: class SimCLRModel(LightningModule):
        def __init__(self, learning_rate: float = 0.001):
            super().__init__()
            self.learning_rate = learning_rate

            self.encoder = ImageEncoder()

```

```

        self.projector = torch.nn.Sequential(
            torch.nn.Linear(2048, 1024),
            torch.nn.ReLU(),
            torch.nn.Linear(1024, 128),
        )

    def configure_optimizers(self):
        optimizer = torch.optim.Adam(self.parameters(), lr=self.learning_rate)
        return optimizer

    def process_batch(self, batch):
        # TASK: Implement the process_batch function
        batch = [self.encoder(x) for x in batch]
        batch = [self.projector(x) for x in batch]
        loss = simclr_loss(batch[0], batch[1])
        return loss

    def training_step(self, batch, batch_idx):
        loss = self.process_batch(batch)
        self.log('train_loss', loss, prog_bar=True)
        if batch_idx == 0:
            grid = torchvision.utils.make_grid(torch.cat((batch[0][0:4, ...],
↪batch[1][0:4, ...])), dim=0), nrow=4, normalize=True)
            self.logger.experiment.add_image('train_images', grid, self.
↪global_step)
        return loss

    def validation_step(self, batch, batch_idx):
        loss = self.process_batch(batch)
        self.log('val_loss', loss, prog_bar=True)

```

Check SimCLR training step. Here you can test that your code runs fine by training the model for 5 epochs using the cell below.

Report the training and validation loss at the end of 5 epochs.

[14]: # DO NOT MODIFY THIS CELL! IT IS FOR CHECKING THE IMPLEMENTATION ONLY.

```

seed_everything(33, workers=True)

data = SimCLRPneumoniaMNISTDataModule(batch_size=32)

model = SimCLRModel()

trainer = Trainer(
    max_epochs=5,
    accelerator='auto',

```

```

    devices=1,
    logger=TensorBoardLogger(save_dir='./lightning_logs/coursework/',
    ↪name='simclr'),
    callbacks=[ModelCheckpoint(monitor='val_loss', mode='min'),
    ↪TQDMProgressBar(refresh_rate=10)],
)
trainer.fit(model=model, datamodule=data)

```

INFO:lightning_fabric.utilities.seed:Seed set to 33

Using downloaded and verified file: ./data/coursework/pneumoniarnist.npz

Using downloaded and verified file: ./data/coursework/pneumoniarnist.npz

Using downloaded and verified file: ./data/coursework/pneumoniarnist.npz

INFO:pytorch_lightning.utilities.rank_zero:GPU available: True (cuda), used: True

INFO:pytorch_lightning.utilities.rank_zero:TPU available: False, using: 0 TPU cores

INFO:pytorch_lightning.utilities.rank_zero:IPU available: False, using: 0 IPUs

INFO:pytorch_lightning.utilities.rank_zero:HPU available: False, using: 0 HPUs

INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]

INFO:pytorch_lightning.callbacks.model_summary:

	Name	Type	Params
0	encoder	ImageEncoder	23.5 M
1	projector	Sequential	2.2 M

```

25.7 M    Trainable params
0         Non-trainable params
25.7 M    Total params
102.925   Total estimated model params size (MB)

```

Sanity Checking: | | 0/? [00:00<?, ?it/s]

/usr/local/lib/python3.10/dist-

packages/pytorch_lightning/trainer/connectors/data_connector.py:441: The 'val_dataloader' does not have many workers which may be a bottleneck. Consider increasing the value of the `num\_workers` argument` to `num_workers=7` in the `DataLoader` to improve performance.

/usr/local/lib/python3.10/dist-

packages/pytorch_lightning/trainer/connectors/data_connector.py:441: The 'train_dataloader' does not have many workers which may be a bottleneck. Consider increasing the value of the `num\_workers` argument` to `num_workers=7` in the `DataLoader` to improve performance.

Training: | | 0/? [00:00<?, ?it/s]

Validation: | | 0/? [00:00<?, ?it/s]

Validation: | | 0/? [00:00<?, ?it/s]

```
Validation: |           | 0/? [00:00<?, ?it/s]
```

```
Validation: |           | 0/? [00:00<?, ?it/s]
```

```
Validation: |           | 0/? [00:00<?, ?it/s]
```

```
INFO:pytorch_lightning.utilities.rank_zero:`Trainer.fit` stopped: `max_epochs=5`  
reached.
```

1.4.3 SimCLR training results

```
train_loss = 3.4913
```

```
val_loss = 3.4966
```

1.5 Part C: Linear probing and model finetuning.

In this part, you are given two different image encoders that were pre-trained with different datasets and training strategies. The objective for this task is to assess the performance of these two encoders in a downstream classification task. To this end, you are asked to implement evaluation routines seen in the lecture: linear probing and model finetuning. The downstream task is the prediction of Pneumonia in the (small) chest X-ray images from the PneumoniaMNIST dataset.

This part can be broken down into the following tasks: 1. Adapt your PneumoniaMNIST dataset for the image classification task. 2. Implement a classification model with a linear layer attached to a pre-trained image encoder. 3. For both pre-trained encoders: - a) Train the classifier on top of the frozen encoder (linear probing) - b) Finetune the entire model (including the encoder). 4. Evaluate all models on the test set, and provide a brief summary (no more than 300 words) with an analysis of your findings.

1.5.1 Task C-1: Adapt your PneumoniaMNIST dataset for the image classification task.

We can base our implementation largely on the `SimCLRPneumoniaMNISTDataset` and adapt it to make it suitable for image classification. Think about a suitable data augmentation pipeline. Check previous tutorials for inspiration.

```
[47]: class PneumoniaMNISTDataset(MedMNIST):  
    def __init__(self, split = 'train', augmentation: bool = False):  
        ''' Dataset class for Pneumonia MNST.  
        The provided init function will automatically download the necessary  
        files at the first class initialisation.  
  
        :param split: 'train', 'val' or 'test', select subset  
  
        '''  
        self.flag = "pneumoniamnist"  
        self.size = 28  
        self.size_flag = ""  
        self.root = './data/coursework/'  
        self.info = INFO[self.flag]
```

```

self.download()

npz_file = np.load(os.path.join(self.root, "pneumoniamnist.npz"))

self.split = split

# Load all the images
assert self.split in ['train', 'val', 'test']

self.imgs = npz_file[f'{self.split}_images']
self.labels = npz_file[f'{self.split}_labels']

self.do_augment = augmentation

# TASK: Define here your data augmentation pipeline suitable for
↪classification.
# Check previous tutorials for inspiration.
self.augmentation_pipeline = v2.Compose([
    v2.RandomResizedCrop(size=28, scale=(0.5, 1.0), antialias=True),
    v2.ColorJitter(brightness=0.15, contrast=None, saturation=None,
↪hue=None),
    v2.RandomRotation(degrees=15),

    v2.ToImage(),
    v2.ToDtype(torch.float32, scale=True),
])

self.non_augmentation_pipeline = v2.Compose([
    v2.ToImage(),
    v2.ToDtype(torch.float32, scale=True),
])

def __len__(self):
    return self.imgs.shape[0]

def __getitem__(self, index):
    # TASK: Implement the __getitem__ function to return the image and its
↪class label.
    label = self.labels[index]
    img = self.imgs[index]
    img = img.reshape((1, 28, 28))
    img = torch.from_numpy(img)
    if self.do_augment:
        img = self.augmentation_pipeline(img)
    else:
        img = self.non_augmentation_pipeline(img)
    return img, label

```

Again, we use a `LightningDataModule` for handling your PneumoniaMNIST dataset. No changes needed for this part.

```
[48]: class PneumoniaMNISTDataModule(LightningDataModule):
    def __init__(self, batch_size: int = 32):
        super().__init__()
        self.batch_size = batch_size
        self.train_set = PneumoniaMNISTDataset(split='train', augmentation=True)
        self.val_set = PneumoniaMNISTDataset(split='val', augmentation=False)
        self.test_set = PneumoniaMNISTDataset(split='test', augmentation=False)

    def train_dataloader(self):
        return DataLoader(dataset=self.train_set, batch_size=self.batch_size,
↪shuffle=True)

    def val_dataloader(self):
        return DataLoader(dataset=self.val_set, batch_size=self.batch_size,
↪shuffle=False)

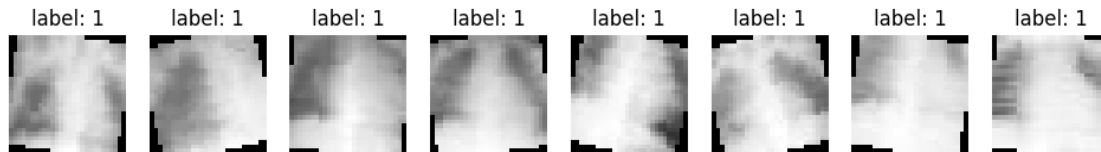
    def test_dataloader(self):
        return DataLoader(dataset=self.test_set, batch_size=self.batch_size,
↪shuffle=False)
```

Check dataset implementation. Run the below cell to visualise a batch of your training dataloader.

```
[49]: # DO NOT MODIFY THIS CELL! IT IS FOR CHECKING THE IMPLEMENTATION ONLY.

# Initialise data module
datamodule = PneumoniaMNISTDataModule()
# Get train dataloader
train_dataloader = datamodule.train_dataloader()
# Get first batch
batch = next(iter(train_dataloader))
# Visualise the images
images, labels = batch
f, ax = plt.subplots(1, 8, figsize=(12,4))
for i in range(8):
    ax[i].imshow(images[i, 0], cmap='gray')
    ax[i].set_title('label: ' + str(labels[i].item()))
    ax[i].axis("off")
```

Using downloaded and verified file: `./data/coursework/pneumoniamnist.npz`
Using downloaded and verified file: `./data/coursework/pneumoniamnist.npz`
Using downloaded and verified file: `./data/coursework/pneumoniamnist.npz`



1.5.2 Task C-2: Implement a classification model with a linear layer attached to a pre-trained image encoder.

We first download the weights of the two pre-trained image encoders. One of them has been trained with the self-supervised SimCLR objective on a large publicly available chest X-ray dataset (different from PneumoniaMNIST). The other encoder is a standard ImageNet backbone that has been trained with a supervised classification objective on the ImageNet dataset.

```
[19]: ! wget https://www.doc.ic.ac.uk/~bglocker/teaching/mli/coursework.zip
      ! unzip coursework.zip

--2024-02-19 20:27:39--
https://www.doc.ic.ac.uk/~bglocker/teaching/mli/coursework.zip
Resolving www.doc.ic.ac.uk (www.doc.ic.ac.uk)... 146.169.13.6
Connecting to www.doc.ic.ac.uk (www.doc.ic.ac.uk)|146.169.13.6|:443...
connected.
HTTP request sent, awaiting response... 200 OK
Length: 191224097 (182M) [application/zip]
Saving to: 'coursework.zip'

coursework.zip      100%[=====>] 182.37M   225MB/s   in 0.8s

2024-02-19 20:27:40 (225 MB/s) - 'coursework.zip' saved [191224097/191224097]

Archive:  coursework.zip
  inflating: data/coursework/model_chestxray.ckpt
  inflating: data/coursework/model_imagenet.ckpt
```

We provide the function for loading the encoders. No changes needed here.

```
[50]: def load_encoder_from_checkpoint(checkpoint_path):
      ckpt = torch.load(checkpoint_path, map_location='cpu')
      simclr_module = SimCLRModel()
      print(simclr_module.load_state_dict(state_dict=ckpt))
      return simclr_module.encoder.eval()

imagenet_model = './data/coursework/model_imagenet.ckpt'
chestxray_model = './data/coursework/model_chestxray.ckpt'
```

```
[51]: model_names = [
        'imagenet',
        'chestxray'
    ]

    def get_ckpt(model_name):
        return fr"./data/coursework/model_{model_name}.ckpt"

    print(get_ckpt(model_names[0]))
```

./data/coursework/model_imagenet.ckpt

Now, implement a classification model as a LightningModule for image classification using a pre-trained image encoder.

The model should have a flag in the init function `freeze_encoder` that if set to true freezes all the weights in the encoder (used for linear probing), and if set to false all weights are trainable (used for model finetuning).

Hint: Check out previous tutorials for inspiration on how to implement a classification model as LightningModule. For the coursework, we recommend using the Area Under the Receiver Operating Characteristic Curve (ROC-AUC) performance metric (instead of accuracy). ROC-AUC is measure of the overall discriminative power of a classification model. You can use the readily available implementation in [torchmetrics](#). You should log the ROC-AUC similar to how we logged accuracy in previous tutorials.

```
[52]: # TASK: Implement the ImageClassifier class
        # Check previous tutorials for insipration how to implement an `ImageClassifier`
        class ImageClassifier(LightningModule):
            def __init__(
                self,
                pretrained_encoder: torch.nn.Module,
                freeze_encoder: bool = True,
                output_dim: int = 2,
                learning_rate: float = 0.001
            ):

                super().__init__()

                self.freeze_encoder = freeze_encoder
                self.output_dim = output_dim
                self.learning_rate = learning_rate

                self.encoder = pretrained_encoder.train(mode=not freeze_encoder)

                self.decoder = torch.nn.Linear(2048, self.output_dim)

            def forward(self, x):
                if self.freeze_encoder:
```

```

        with torch.no_grad():
            x = self.encoder(x)
        else:
            x = self.encoder(x)
        x = x.view(x.size(0), -1)
        return self.decoder(x), x

def configure_optimizers(self):
    optimizer = torch.optim.Adam(self.parameters(), lr=self.learning_rate)
    return optimizer

def process_batch(self, batch):
    x, y = batch
    y = y.squeeze(dim=1)
    logits, xt = self(x)
    loss = F.cross_entropy(logits, y)
    probs = torch.softmax(logits, dim=1)
    preds = torch.argmax(probs, dim=1)
    acc = auroc(probs, y, task='multiclass', num_classes=self.output_dim)
    return loss, acc, xt

def training_step(self, batch, batch_idx):
    loss, acc, xt = self.process_batch(batch)
    self.log('train_loss', loss, prog_bar=True)
    self.log('train_auroc', acc, prog_bar=True)
    return loss

def validation_step(self, batch, batch_idx):
    loss, acc, _ = self.process_batch(batch)
    self.log('val_loss', loss, prog_bar=True)
    self.log('val_auroc', acc, prog_bar=True)

def test_step(self, batch, batch_idx):
    loss, acc, xt = self.process_batch(batch)
    self.log('test_loss', loss)
    self.log('test_auroc', acc)

```

1.5.3 Task C-3a: Implement training and testing for linear probing.

Train two classification models using linear probing, one for each of the two provided image encoders. Evaluate on both the validation and test sets.

Note: Training for 25 epochs should be sufficient.

```

[58]: def train_and_test(mode):

        seed_everything(33, workers=True)

```

```

data = PneumoniaMNISTDataModule(batch_size=32)
# TASK: Implement the linear probing training and testing routines.

modes = [
    'finetuning',
    'linear-probing'
]

assert(mode in modes)

freeze_encoder = mode == modes[1]

print()
print('we freeze encoder' if freeze_encoder else 'we do NOT freeze encoder')

for i in range(len(model_names)):

    model_name = model_names[i]

    print(model_name)
    print()

    model = ImageClassifier(
        pretrained_encoder=load_encoder_from_checkpoint(get_ckpt(model_name)),
        freeze_encoder=freeze_encoder,
    )

    trainer = Trainer(
        max_epochs=25,
        accelerator='auto',
        devices=1,
        logger=TensorBoardLogger(save_dir='./lightning_logs/coursework/',
        ↪name=f'image-classifier-{mode}-{model_name}'),
        callbacks=[ModelCheckpoint(monitor='val_loss', mode='min'),
        ↪TQDMProgressBar(refresh_rate=10)],
    )
    trainer.fit(model=model, datamodule=data)

    trainer.validate(model=model, datamodule=data, ckpt_path=trainer.
    ↪checkpoint_callback.best_model_path)

    trainer.test(model=model, datamodule=data, ckpt_path=trainer.
    ↪checkpoint_callback.best_model_path)

    print()

print(models)

```

```
<module 'torchvision.models' from '/usr/local/lib/python3.10/dist-  
packages/torchvision/models/__init__.py'>
```

```
[59]: train_and_test(mode='linear-probing')
```

```
INFO:lightning_fabric.utilities.seed:Seed set to 33
```

```
Using downloaded and verified file: ./data/coursework/pneumoniarnist.npz
```

```
Using downloaded and verified file: ./data/coursework/pneumoniarnist.npz
```

```
Using downloaded and verified file: ./data/coursework/pneumoniarnist.npz
```

```
we freeze encoder
```

```
imagenet
```

```
INFO:pytorch_lightning.utilities.rank_zero:GPU available: True (cuda), used:  
True
```

```
INFO:pytorch_lightning.utilities.rank_zero:TPU available: False, using: 0 TPU  
cores
```

```
INFO:pytorch_lightning.utilities.rank_zero:IPU available: False, using: 0 IPUs
```

```
INFO:pytorch_lightning.utilities.rank_zero:HPU available: False, using: 0 HPUs
```

```
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES:  
[0]
```

```
INFO:pytorch_lightning.callbacks.model_summary:
```

```
  | Name      | Type           | Params  
-----
```

```
0 | encoder    | ImageEncoder   | 23.5 M
```

```
1 | decoder    | Linear         | 4.1 K  
-----
```

```
23.5 M    Trainable params
```

```
0          Non-trainable params
```

```
23.5 M    Total params
```

```
94.023     Total estimated model params size (MB)
```

```
<All keys matched successfully>
```

```
Sanity Checking: |           | 0/? [00:00<?, ?it/s]
```

```
/usr/local/lib/python3.10/dist-
```

```
packages/pytorch_lightning/trainer/connectors/data_connector.py:441: The  
'val_dataloader' does not have many workers which may be a bottleneck. Consider  
increasing the value of the `num_workers` argument` to `num_workers=7` in the  
`DataLoader` to improve performance.
```

```
/usr/local/lib/python3.10/dist-
```

```
packages/pytorch_lightning/trainer/connectors/data_connector.py:441: The  
'train_dataloader' does not have many workers which may be a bottleneck.
```

```
Consider increasing the value of the `num_workers` argument` to `num_workers=7`  
in the `DataLoader` to improve performance.
```

```
Training: |           | 0/? [00:00<?, ?it/s]
```

```
Validation: |           | 0/? [00:00<?, ?it/s]
```



```

path at ./lightning_logs/coursework/image-classifier-linear-probing-
imagenet/version_2/checkpoints/epoch=20-step=3108.ckpt
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES:
[0]
INFO:pytorch_lightning.utilities.rank_zero:Loaded model weights from the
checkpoint at ./lightning_logs/coursework/image-classifier-linear-probing-
imagenet/version_2/checkpoints/epoch=20-step=3108.ckpt
Validation: |           | 0/? [00:00<?, ?it/s]

```

Validate metric	DataLoader 0
val_auroc	0.9529308080673218
val_loss	0.23169033229351044

```

INFO:pytorch_lightning.utilities.rank_zero:Restoring states from the checkpoint
path at ./lightning_logs/coursework/image-classifier-linear-probing-
imagenet/version_2/checkpoints/epoch=20-step=3108.ckpt
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES:
[0]
INFO:pytorch_lightning.utilities.rank_zero:Loaded model weights from the
checkpoint at ./lightning_logs/coursework/image-classifier-linear-probing-
imagenet/version_2/checkpoints/epoch=20-step=3108.ckpt
/usr/local/lib/python3.10/dist-
packages/pytorch_lightning/trainer/connectors/data_connector.py:441: The
'test_dataloader' does not have many workers which may be a bottleneck. Consider
increasing the value of the `num_workers` argument` to `num_workers=7` in the
`DataLoader` to improve performance.
Testing: |           | 0/? [00:00<?, ?it/s]

```

Test metric	DataLoader 0
test_auroc	0.9283431768417358
test_loss	0.3568597137928009

chestxray

```

INFO:pytorch_lightning.utilities.rank_zero:GPU available: True (cuda), used:
True
INFO:pytorch_lightning.utilities.rank_zero:TPU available: False, using: 0 TPU
cores
INFO:pytorch_lightning.utilities.rank_zero:IPU available: False, using: 0 IPUs

```

```
INFO:pytorch_lightning.utilities.rank_zero:HPU available: False, using: 0 HPUs
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES:
[0]
```

```
INFO:pytorch_lightning.callbacks.model_summary:
```

	Name	Type	Params
0	encoder	ImageEncoder	23.5 M
1	decoder	Linear	4.1 K

23.5 M	Trainable params
0	Non-trainable params
23.5 M	Total params
94.023	Total estimated model params size (MB)

```
<All keys matched successfully>
```

```
Sanity Checking: |           | 0/? [00:00<?, ?it/s]
```

```
Training: |           | 0/? [00:00<?, ?it/s]
```

```
Validation: |           | 0/? [00:00<?, ?it/s]
```

```
Validation: |           | 0/? [00:00<?, ?it/s]
```

```
Validation: |           | 0/? [00:00<?, ?it/s]
```

```
Validation: |           | 0/? [00:00<?, ?it/s]
```

```
Validation: |           | 0/? [00:00<?, ?it/s]
```

```
Validation: |           | 0/? [00:00<?, ?it/s]
```

```
Validation: |           | 0/? [00:00<?, ?it/s]
```

```
Validation: |           | 0/? [00:00<?, ?it/s]
```

```
Validation: |           | 0/? [00:00<?, ?it/s]
```

```
Validation: |           | 0/? [00:00<?, ?it/s]
```

```
Validation: |           | 0/? [00:00<?, ?it/s]
```

```
Validation: |           | 0/? [00:00<?, ?it/s]
```

```
Validation: |           | 0/? [00:00<?, ?it/s]
```

```
Validation: |           | 0/? [00:00<?, ?it/s]
```

```
Validation: |           | 0/? [00:00<?, ?it/s]
```

```
Validation: |           | 0/? [00:00<?, ?it/s]
```

```
Validation: |           | 0/? [00:00<?, ?it/s]
```

```
Validation: |           | 0/? [00:00<?, ?it/s]
```

```
Validation: |           | 0/? [00:00<?, ?it/s]
```

```
Validation: |           | 0/? [00:00<?, ?it/s]
```

Validation: | | 0/? [00:00<?, ?it/s]

Validation: | | 0/? [00:00<?, ?it/s]

Validation: | | 0/? [00:00<?, ?it/s]

Validation: | | 0/? [00:00<?, ?it/s]

Validation: | | 0/? [00:00<?, ?it/s]

INFO:pytorch_lightning.utilities.rank_zero:`Trainer.fit` stopped:
`max\_epochs=25` reached.

INFO:pytorch_lightning.utilities.rank_zero:Restoring states from the checkpoint
path at ./lightning_logs/coursework/image-classifier-linear-probing-
chestxray/version_2/checkpoints/epoch=22-step=3404.ckpt

INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES:
[0]

INFO:pytorch_lightning.utilities.rank_zero:Loaded model weights from the
checkpoint at ./lightning_logs/coursework/image-classifier-linear-probing-
chestxray/version_2/checkpoints/epoch=22-step=3404.ckpt

Validation: | | 0/? [00:00<?, ?it/s]

Validate metric

DataLoader 0

val_auroc

0.9157589077949524

val_loss

0.3132123351097107

INFO:pytorch_lightning.utilities.rank_zero:Restoring states from the checkpoint
path at ./lightning_logs/coursework/image-classifier-linear-probing-
chestxray/version_2/checkpoints/epoch=22-step=3404.ckpt

INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES:
[0]

INFO:pytorch_lightning.utilities.rank_zero:Loaded model weights from the
checkpoint at ./lightning_logs/coursework/image-classifier-linear-probing-
chestxray/version_2/checkpoints/epoch=22-step=3404.ckpt

Testing: | | 0/? [00:00<?, ?it/s]

Test metric

DataLoader 0

test_auroc

0.8647798299789429

test_loss

0.47509267926216125

1.5.4 Task C-3b: Implement training and testing for model finetuning.

Repeat the experiments, but this time using model finetuning instead of linear probing. Evaluate on both the validation and test sets.

```
[60]: train_and_test(mode='finetuning')
```

```
INFO:lightning_fabric.utilities.seed:Seed set to 33

Using downloaded and verified file: ./data/coursework/pneumoniarnist.npz
Using downloaded and verified file: ./data/coursework/pneumoniarnist.npz
Using downloaded and verified file: ./data/coursework/pneumoniarnist.npz

we do NOT freeze encoder
imagenet

INFO:pytorch_lightning.utilities.rank_zero:GPU available: True (cuda), used:
True
INFO:pytorch_lightning.utilities.rank_zero:TPU available: False, using: 0 TPU
cores
INFO:pytorch_lightning.utilities.rank_zero:IPU available: False, using: 0 IPUs
INFO:pytorch_lightning.utilities.rank_zero:HPU available: False, using: 0 HPUs
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES:
[0]
INFO:pytorch_lightning.callbacks.model_summary:
  | Name      | Type           | Params
-----
0 | encoder    | ImageEncoder   | 23.5 M
1 | decoder    | Linear         | 4.1 K
-----
23.5 M    Trainable params
0         Non-trainable params
23.5 M    Total params
94.023    Total estimated model params size (MB)
<All keys matched successfully>

Sanity Checking: |           | 0/? [00:00<?, ?it/s]

Training: |           | 0/? [00:00<?, ?it/s]

Validation: |           | 0/? [00:00<?, ?it/s]

Validation: |           | 0/? [00:00<?, ?it/s]

Validation: |           | 0/? [00:00<?, ?it/s]

Validation: |           | 0/? [00:00<?, ?it/s]

Validation: |           | 0/? [00:00<?, ?it/s]

Validation: |           | 0/? [00:00<?, ?it/s]

Validation: |           | 0/? [00:00<?, ?it/s]
```

[illegible]

```
INFO:pytorch_lightning.utilities.rank_zero:`Trainer.fit` stopped:
`max_epochs=25` reached.
```

```
INFO:pytorch_lightning.utilities.rank_zero:Restoring states from the checkpoint
path at ./lightning_logs/coursework/image-classifier-finetuning-
imagenet/version_1/checkpoints/epoch=15-step=2368.ckpt
```

```
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]
```

```
INFO:pytorch_lightning.utilities.rank_zero:Loaded model weights from the
checkpoint at ./lightning_logs/coursework/image-classifier-finetuning-
imagenet/version_1/checkpoints/epoch=15-step=2368.ckpt
```

```
Validation: |          | 0/? [00:00<?, ?it/s]
```

Validate metric

DataLoader 0

```
val_auroc
```

0.9799266457557678

```
val_loss
```

0.20239220559597015

```
INFO:pytorch_lightning.utilities.rank_zero:Restoring states from the checkpoint
path at ./lightning_logs/coursework/image-classifier-finetuning-
imagenet/version_1/checkpoints/epoch=15-step=2368.ckpt
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES:
[0]
INFO:pytorch_lightning.utilities.rank_zero:Loaded model weights from the
checkpoint at ./lightning_logs/coursework/image-classifier-finetuning-
imagenet/version_1/checkpoints/epoch=15-step=2368.ckpt
Testing: |           | 0/? [00:00<?, ?it/s]
```

Test metric	DataLoader 0
test_auroc	0.9630464315414429
test_loss	0.23967616260051727

chestxray

```
INFO:pytorch_lightning.utilities.rank_zero:GPU available: True (cuda), used:
True
INFO:pytorch_lightning.utilities.rank_zero:TPU available: False, using: 0 TPU
cores
INFO:pytorch_lightning.utilities.rank_zero:IPU available: False, using: 0 IPUs
INFO:pytorch_lightning.utilities.rank_zero:HPU available: False, using: 0 HPUs
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES:
[0]
```

```
INFO:pytorch_lightning.callbacks.model_summary:
```

	Name	Type	Params
0	encoder	ImageEncoder	23.5 M
1	decoder	Linear	4.1 K

```

23.5 M    Trainable params
0         Non-trainable params
23.5 M    Total params
94.023    Total estimated model params size (MB)
```

<All keys matched successfully>

```
Sanity Checking: |           | 0/? [00:00<?, ?it/s]
```

```
Training: |           | 0/? [00:00<?, ?it/s]
```

```
Validation: |           | 0/? [00:00<?, ?it/s]
```

```
Validation: |           | 0/? [00:00<?, ?it/s]
```

```
Validation: |           | 0/? [00:00<?, ?it/s]
```

[illegible]

```
INFO:pytorch_lightning.utilities.rank_zero:`Trainer.fit` stopped:  
`max_epochs=25` reached.
```

```
INFO:pytorch_lightning.utilities.rank_zero:Restoring states from the checkpoint
path at ./lightning_logs/coursework/image-classifier-finetuning-
chestxray/version_1/checkpoints/epoch=24-step=3700.ckpt
```

```
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES:
[0]
```

```
INFO:pytorch_lightning.utilities.rank_zero:Loaded model weights from the
checkpoint at ./lightning_logs/coursework/image-classifier-finetuning-
chestxray/version_1/checkpoints/epoch=24-step=3700.ckpt
```

```
Validation: |          | 0/? [00:00<?, ?it/s]
```

Validate metric

DataLoader 0

```
val_auroc          0.9899923205375671
val_loss           0.11203823238611221
```

```
INFO:pytorch_lightning.utilities.rank_zero:Restoring states from the checkpoint
path at ./lightning_logs/coursework/image-classifier-finetuning-
chestxray/version_1/checkpoints/epoch=24-step=3700.ckpt
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES:
[0]
INFO:pytorch_lightning.utilities.rank_zero:Loaded model weights from the
checkpoint at ./lightning_logs/coursework/image-classifier-finetuning-
chestxray/version_1/checkpoints/epoch=24-step=3700.ckpt
Testing: |          | 0/? [00:00<?, ?it/s]
```

Test metric	DataLoader 0
test_auroc	0.9672763347625732
test_loss	0.24408504366874695

1.5.5 Task C-4: Your evaluation report.

Provide a brief summary (no more than 300 words) with an analysis of your findings. Try explaining the observed performance.

AUC-ROC scores:

Linear probing: 1. ImageNet: 0.928 2. Chest x-ray: 0.865

Finetuning 1. ImageNet: 0.963 2. Chest x-ray: 0.967

All experiment runs yield a fairly high AUC-ROC score ($> 85\%$). One reason behind that is that the chosen binary classification task is relatively easy, esp. when compared with ImageNet, which has 1,000 classes.

In the original SimCLR paper the supervised ResNet model consistently beats SimCLR in terms of accuracy assuming both models have the same number of parameters. Here, that's also the case but only in linear probing - in finetuning the models have a similar performance. The superiority of ImageNet over SimCLR in the linear probing task is interesting given that ImageNet encoder has never trained on a chest x-ray. ImageNet has the advantage of being supervised and having been pre-trained on a larger and more diverse dataset than SimCLR, which could provide a more generalised feature representation. This allows it to perform well even when applied to this new domain, without needing to adjust its weights significantly.

Finetuning performed better than linear probing for both ImageNet and SimCLR which suggests that allowing pre-trained models to adjust their weights (finetuning) rather than keeping them

frozen (linear probing) leads to better performance when adapted to a new task. This is particularly evident in the case of ImageNet when despite being trained on a different data set than chest x-rays with finetuning it achieves high AUC-ROC scores.

In finetuning both models have a similar performance. This suggests that when both models are able to adjust their weights to the specific features of the chest x-ray images, the advantages of being pre-trained on a broader data-set are diminished.

The test loss values are consistent with AUC-ROC scores, i.e. the lower the loss value, the higher the AUC-ROC score.

1.6 Logging

```
[61]: %load_ext tensorboard
      %tensorboard --logdir './lightning_logs/coursework/'
```

The tensorboard extension is already loaded. To reload it, use:

```
%reload_ext tensorboard
```

<IPython.core.display.Javascript object>